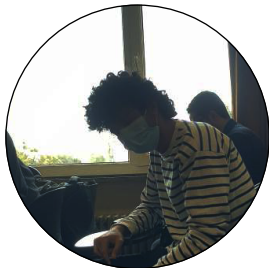


Robotics and Computer Vision

Final Project: Face Recognition Attendance System

TriCortex Perception Team



Hamed Nabipour
40222153



Sina Hasanpour
40216723



**Amir Mohamad
Ahmadi**
40214403

فهرست مطالب

۳	۱	مقدمه و شرح مسئله
۳	۱.۱	نوآوری‌های اصلی سیستم
۳	۲	معماری سامانه و خط لوله پردازش
۴	۱.۲	دیاگرام معماری سامانه
۴	۳	لایه‌ی بینایی ماشین: بردار چهره و تشخیص هویت
۴	۱.۳	آشکارسازی و کدگذاری چهره با InsightFace/ArcFace
۵	۲.۳	استخراج بردار از فریم خام
۵	۳.۳	مقایسه با بردارهای ذخیره‌شده (Cosine Similarity)
۶	۴.۳	ذخیره‌سازی بردارها در دیتابیس
۶	۴	لایه‌ی داده: پایگاه داده و منطق حضور و غیاب
۶	۱.۴	مدل‌های ORM و پایگاه داده
۷	۲.۴	منطق ثبت ورود و خروج
۷	۳.۴	تفکیک مهمان از کارمند
۷	۵	لایه‌ی وب: FastAPI، داشبورد و امنیت
۷	۱.۵	معماری FastAPI و رفع مسدودیت Event Loop
۷	۲.۵	پخش زنده‌ی تصویر با MJPEG
۸	۳.۵	امنیت: JWT و RBAC
۸	۶	ماژول‌های بینایی ماشین: کیفیت، زنده‌بودن و ردیابی
۸	۱.۶	ارزیابی کیفیت تصویر (Quality)
۸	۲.۶	تشخیص زنده‌بودن (Liveness)
۸	۳.۶	ردیابی چهره (Tracking)
۹	۷	تصاویر محیط برنامه
۱۰	۸	روند تیمی و تقسیم کار
۱۰	۱.۸	فاز صفر — طراحی معماری (کار مشترک)
۱۰	۲.۸	فاز یک — سینا حسن‌پور (داده و منطق)
۱۰	۳.۸	فاز دو — حامد نبی‌پور (بینایی ماشین و هوش مصنوعی)
۱۰	۴.۸	فاز سه — امیرمحمد احمدی (وب، API و رابط کاربری)
۱۰	۹	نتیجه‌گیری و کارهای آینده

۱ مقدمه و شرح مسئله

در این پروژه، یک سامانه‌ی جامع حضور و غیاب خودکار مبتنی بر تشخیص چهره (Face Recognition Atten-dance System) پیاده‌سازی شده است. در سامانه‌های سنتی، ثبت ورود و خروج کارکنان با کارت، انگشت یا ورود دستی انجام می‌شود که مستعد خطای انسانی، جعل هویت و اتلاف زمان است. سامانه‌ی طراحی‌شده در این پروژه، با تحلیل تصویر زنده‌ی دوربین وب‌کم، هویت افراد را به‌صورت خودکار و بلادرنگ تشخیص می‌دهد و بدون نیاز به تماس فیزیکی، ساعت ورود و خروج آن‌ها را ثبت می‌کند. چالش‌های اصلی این پروژه عبارت‌اند از:

۱. شناسایی دقیق هویت اشخاص در شرایط نوری و زاویه‌ی متفاوت.
 ۲. جلوگیری از تقلب با عکس چاپی یا صفحه‌ی موبایل (تشخیص زنده‌بودن).
 ۳. ثبت حضور پایدار و بدون رکوردهای تکراری در طول روز.
 ۴. ارائه‌ی یک رابط وب بلادرنگ برای مانیتورینگ و مدیریت.
- در ادامه، ابتدا معماری کلی سامانه و خط لوله‌ی پردازش تصویر توضیح داده می‌شود. سپس پایگاه داده، منطق حضور و غیاب، رابط وب و در نهایت بهینه‌سازی‌های انجام‌شده جهت دستیابی به عملکرد بلادرنگ بررسی می‌گردد.

۱.۱ نوآوری‌های اصلی سیستم

- استفاده از بردار ۵۱۲ بُعدی ArcFace به‌جای بردار ۱۲۸ بُعدی سنتی جهت افزایش دقت تفکیک هویت.
- تشخیص زنده‌بودن (Liveness) سه‌لایه برای جلوگیری از تهاجم چهره‌ای.
- ردیابی هویت با Kalman Filter و رأی‌گیری در طول زمان برای پایداری تشخیص.
- تفکیک چهره‌های ناشناس به‌عنوان «مهمان» و جلوگیری از ثبت حضور کاذب.

۲ معماری سامانه و خط لوله پردازش

سامانه‌ی طراحی‌شده از سه لایه‌ی اصلی تشکیل شده است:

۱. لایه‌ی بینایی ماشین: آشکارسازی، استخراج بردار چهره، تشخیص کیفیت، تشخیص زنده‌بودن و ردیابی.
 ۲. لایه‌ی داده و منطق: پایگاه داده‌ی رابطه‌ای، مدل‌های ORM و منطق ثبت ورود/خروج.
 ۳. لایه‌ی وب: FastAPI، داشبورد بلادرنگ، Streaming MJPEG، پنل مدیریت و امنیت با JWT/RBAC.
- خط لوله‌ی پردازش تصویر در فایل `face_recognizer.py` به‌صورت زیر تعریف شده است:

```
1 Pipeline:
2 Frame -> RetinaFace detect -> Quality gate -> ArcFace encode ->
3 Match DB -> Liveness check -> Kalman tracking -> Augmentation
```

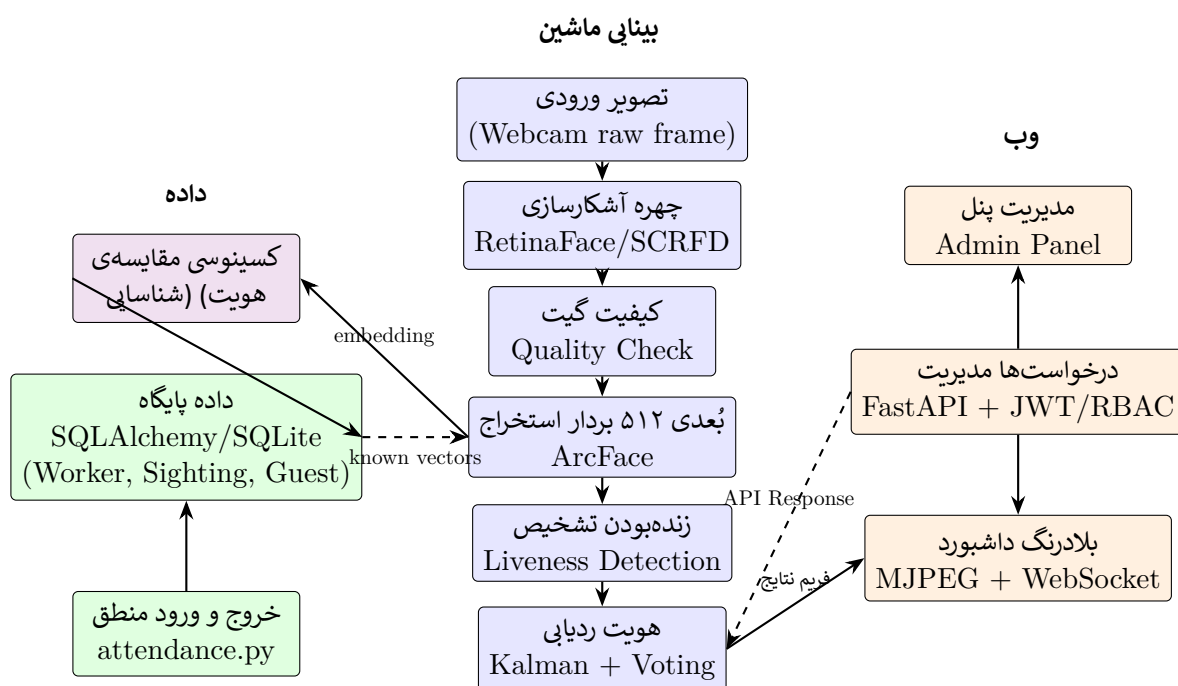
شرح مراحل:

- RetinaFace detect: آشکارسازی موقعیت چهره در فریم.
- Quality gate: بررسی کیفیت تصویر (تاری، نور، زاویه) قبل از پردازش سنگین.
- ArcFace encode: استخراج بردار ۵۱۲ عددی از ناحیه‌ی چهره.
- Match DB: مقایسه‌ی کسینوسی با بردارهای ذخیره‌شده در دیتابیس.

- Liveness check: تشخیص زنده بودن فرد و رد عکس/صفحه‌ی موبایل.
- Kalman tracking: پایدارسازی هویت بین فریم‌های متوالی.
- Augmentation: افزودن نمونه‌ی جدید بردار برای بهبود دقت بلندمدت.

۱.۲ دیاگرام معماری سامانه

شکل ۱ نمای کلی معماری سامانه و جریان داده بین اجزا را نشان می‌دهد. مسیر اصلی پردازش (خط لوله‌ی بینایی) در ستون میانی به صورت عمودی و لایه‌های داده و وب به صورت جانبی قرار گرفته‌اند:



شکل ۱: معماری سامانه: خط لوله‌ی بینایی، لایه‌ی داده و لایه‌ی وب

توضیح جریان: فریم خام از دوربین گرفته می‌شود، چهره توسط SCRFD آشکار و کیفیت آن گیت می‌شود. بردار ۵۱۲ بُعدی ArcFace ساخته و با بردارهای ذخیره شده در دیتابیس با شباهت کسینوسی مقایسه می‌شود (شناسایی هویت). سپس تشخیص زنده بودن و ردیابی با Kalman اعمال می‌شود و نتیجه‌ی نهایی در لایه‌ی داده (ورود/خروج) و لایه‌ی وب (داشبورد بلادرنگ) ثبت و نمایش داده می‌شود.

۳ لایه‌ی بینایی ماشین: بردار چهره و تشخیص هویت

۱.۳ آشکارسازی و کدگذاری چهره با InsightFace/ArcFace

نقطه‌ی قلب این پروژه، بردار چهره یا Embedding است. ایده‌ی اصلی این است که مدل آموزش دیده‌ی ArcFace تصویر یک چهره را دریافت و یک بردار با ۵۱۲ عدد (512-dim) خروجی می‌دهد. این بردار «امضای ریاضی» آن چهره است؛ چهره‌های یک فرد، بردارهای نزدیک به هم و چهره‌های افراد مختلف، بردارهای دور از هم دارند. بارگذاری مدل در سازنده‌ی کلاس FaceRecognizer انجام می‌شود:

```
1 from insightface.app import FaceAnalysis as InsightFaceAnalysis
2 self._insightface_app = InsightFaceAnalysis(
3     name="buffalo_l",
4     providers=["CPUExecutionProvider"],
```

```
5     allowed_modules=["detection", "recognition"],  
6 )  
7 det = int(insightface_det_size)  
8 self._insightface_app.prepare(ctx_id=-1, det_size=(det, det))
```

- name="buffalo_l": بسته‌ی مدل آماده‌ی InsightFace.
- providers=["CPUExecutionProvider"]: اجرای مدل روی CPU بدون نیاز به GPU.
- allowed_modules=["detection", "recognition"]: فقط دو ماژول لازم (آشکارسازی و کدگذاری) فعال شده تا سرعت بالاتر رود.
- det_size=(det, det): اندازه‌ی ورودی آشکارساز باید مربع باشد؛ در این پروژه det=320 در نظر گرفته شد.

نکته فنی مهم: رفع خطای broadcast در SCRFD

در نسخه‌های نخستین، اندازه‌ی ورودی به‌صورت (320, 240) تنظیم شده بود که باعث خطای broadcast در آشکارساز SCRFD می‌شد. دلیل آن است که SCRFD شبکه‌ی نقاط لنگر (anchor) خود را از ارتفاع ورودی برای هر دو محور x و y می‌سازد و بنابراین اندازه‌ی ورودی باید مربع باشد. با اصلاح به det_size=(320, 320) این مشکل برطرف شده و خط لوله بدون خطا اجرا گردید.

۲.۳ استخراج بردار از فریم خام

برای استخراج بردار باکیفیت، بین فریم خام (raw_frame) بدون هیچ حاشیه‌نویسی و فریم نمایشی (که کادر و متن روی آن کشیده شده) تفکیک صورت گرفته است. اگر بردار از فریم نمایشی استخراج می‌شد، متن‌های روی کادر هم وارد بردار می‌شدند و دقت را خراب می‌کردند. کد زیر بردار چهره را استخراج و نرمال‌سازی می‌کند:

```
1 dets = self._insightface_app.get(frame)  
2 encoding = det.embedding # 512-dim embedding  
3 encoding_norm = encoding / (np.linalg.norm(encoding) + 1e-8)
```

۳.۳ مقایسه با بردارهای ذخیره‌شده (Cosine Similarity)

برای یافتن «این چهره کیست؟»، بردار استخراج‌شده با همه‌ی بردارهای ذخیره‌شده در دیتابیس مقایسه می‌شود. مقایسه با شباهت کسینوسی انجام می‌شود؛ چون هر دو بردار نرمال هستند، حاصل ضرب ماتریسی همان کسینوسی است:

```
1 similarities = known_matrix @ encoding_norm # cosine similarity  
2 best_idx = int(np.argmax(similarities))  
3 best_sim = float(similarities[best_idx])  
4 if best_sim > 1.0 - self.tolerance: # threshold  
5     person_id = match_ids[best_idx] # identity found
```

آستانه‌ی تصمیم‌گیری با tolerance کنترل می‌شود: tolerance - 0.1 با tolerance=0.6، حداقل شباهت پذیرفته‌شده مقدار 0.4 است.

محاسبه‌ی درجه اطمینان (Confidence)

علاوه بر شناسایی، یک درجه اطمینان محاسبه می‌شود که اختلاف بین بهترین و دومین شباهت را در نظر می‌گیرد:

```
1 sd = np.sort(similarities)[::-1]
2 margin = float(sd[0] - sd[1])
3 confidence = best_sim * min(1.0, 0.5 + margin)
4 confidence *= 0.5 + 0.5 * quality_score
```

اگر دو چهره خیلی شبیه باشند (مارجین کوچک)، اطمینان پایین می‌آید و در کیفیت تصویر نیز ضرب می‌شود؛ نتیجه در گیت نهایی پذیرش قرار می‌گیرد.

۴.۳ ذخیره‌سازی بردارها در دیتابیس

در زمان ثبت نام، بردارهای استخراج شده به صورت JSON در جدول face_encodings ذخیره می‌شوند. هر کارمند می‌تواند چند بردار (از زوایا و نورهای متفاوت) داشته باشد تا نرخ بازشناسی بالا باشد:

```
1 for i, enc in enumerate(encodings):
2     fe = FaceEncoding(
3         worker_id=worker.id,
4         encoding=enc.tolist(),          # numpy -> list -> JSON
5         quality_score="0.5",
6     )
7     db.add(fe)
8 db.commit()
```

همچنین رابطه‌ی "delete-orphan" cascade="all" تضمین می‌کند که با حذف یک کارمند، بردارهایش هم خودکار حذف شوند.

۴ لایه‌ی داده: پایگاه داده و منطق حضور و غیاب

۱.۴ مدل‌های ORM و پایگاه داده

پایگاه داده با SQLAlchemy (به عنوان ORM) پیاده‌سازی شده است. مدل‌های اصلی در پوشه‌ی models/ تعریف شده‌اند:

- Worker: مشخصات هر کارمند (شناسه، نام، وضعیت فعال بودن).
- FaceEncoding: بردار ۵۱۲ عددی هر نمونه‌ی چهره.
- FaceSighting: رکورد حضور روزانه (اولین و آخرین دیده شدن).
- Guest: چهره‌های ناشناسی که به عنوان مهمان ثبت شده‌اند.

نمونه‌ای از تعریف مدل کارمند:

```
1 class Worker(Base):
2     __tablename__ = "workers"
3     id = Column(GUID(), primary_key=True, default=uuid.uuid4)
4     worker_id = Column(String(10), unique=True, nullable=False) # e.g.
5     W001
6     name = Column(String(100), nullable=False)
7     is_active = Column(Integer, default=1) # soft delete support
```

دیتابیس پیش فرض SQLite است (فایل cv_attendance.db) اما کد با PostgreSQL نیز سازگار است.

۲.۴ منطق ثبت ورود و خروج

مفهوم کلیدی: اولین دیده شدن در روز = ورود و آخرین دیده شدن = خروج. ساعت کاری نیز از تفاضل این دو محاسبه می شود. این منطق در attendance.py پیاده سازی شده است:

```
1 if sighting is None:
2     sighting = FaceSighting(
3         worker_id=worker.id,
4         event_date=now,
5         first_seen=now,
6         last_seen=now,
7     )
8     db.add(sighting)
9 else:
10     sighting.last_seen = now
11 db.commit()
```

به این ترتیب نیازی به تعامل کاربر با دستگاه (لمس یا دکمه) نیست و صرفاً حضور در دید دوربین، ورود/خروج ثبت می شود.

۳.۴ تفکیک مهمان از کارمند

یکی از باگ های مهمی که در این پروژه کشف و رفع شد، ثبت حضور کاذب برای چهره های ناشناس بود. راه حل: چهره ی ناشناس به عنوان مهمان (Guest) ذخیره می شود ولی حضور او ثبت نمی شود. در main_fastapi.py:

```
1 # Guests are recognized but have no attendance record
2 if not pid.startswith("W"):
3     continue
```

همچنین برای جلوگیری از رکوردهای تکراری، یک Cooldown تعریف شده است؛ اگر یک نفر طی چند ثانیه ی اخیر ثبت شده باشد، دوباره ثبت نمی شود.

۵ لایه ی وب: FastAPI، داشبورد و امنیت

۱.۵ معماری FastAPI و رفع مسدودیت Event Loop

سرویس وب با FastAPI پیاده سازی شده است. یک چالش مهم این بود که کارهای دیتابیزی مسدودکننده هستند و اگر مستقیماً در Event Loop اجرا شوند، کل سرور قفل می شود و مرورگر timeout می خورد. راه حل، تعریف اندپوینت ها به صورت def معمولی است تا FastAPI آن ها را در Thread Pool اجرا کند:

```
1 @app.get("/api/dashboard", tags=["Dashboard"])
2 def api_dashboard():
3     """Declared sync on purpose: blocking DB work runs in the
4     threadpool instead of stalling the event loop."""
5     ...
```

نتیجه ی این بهینه سازی، کاهش زمان پاسخ API به 55-89 ms است.

۲.۵ پخش زنده ی تصویر با MJPEG

تصویر زنده ی دوربین با روش MJPEG (ارسال پشت سرهم فریم های JPEG) در مرورگر نمایش داده می شود:

```
1 _, buf = cv2.imencode(".jpg", display, [cv2.IMWRITE_JPEG_QUALITY, 85])
2 data = buf.tobytes()
3 yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + data + b"\r\n"
```

کیفیت 85 انتخاب شده تا تعادل مناسبی بین کیفیت و حجم برقرار باشد.

۳.۵ امنیت: RBAC و JWT

برای کنترل دسترسی، از دو سازوکار استفاده شده است:

- JWT: پس از ورود، یک توکن رمزنگاری شده صادر می‌شود که اعتبار آن با کلید مخفی امضا می‌شود.
- RBAC: هر کاربر یک «نقش» دارد (ادمین، مدیر، اپراتور، بیننده) و هر نقش مجموعه‌ای از اجازه‌ها دارد. مثلاً فقط ADMIN می‌تواند کارمند حذف کند و VIEWER فقط مشاهده می‌کند.

۶ مازول‌های بینایی ماشین: کیفیت، زنده‌بودن و ردیابی

۱.۶ ارزیابی کیفیت تصویر (Quality)

قبل از کدگذاری چهره، کیفیت تصویر بررسی می‌شود تا از استخراج بردارهای بی‌کیفیت جلوگیری شود. معیارها:

- تاری (Blur): با واریانس Laplacian سنجیده می‌شود؛ تصویر واضح لبه‌های تیز دارد.
- روشنایی (Brightness): میانگین روشنایی پیکسل‌ها.
- کنتراست (Contrast): انحراف معیار پیکسل‌ها.
- زاویه‌ی سر (Pose): چرخش سر به چپ/راست و بالا/پایین از روی نقاط کلیدی.
- پوشیدگی (Occlusion): میزان پوشیده‌شدن صورت.

نمره‌ی نهایی وزن‌دار است و با `quality_score` به‌عنوان گیت کیفیت وارد خط لوله می‌شود.

۲.۶ تشخیص زنده‌بودن (Liveness)

برای جلوگیری از تقلب با عکس یا صفحه‌ی موبایل، سه نشانه ترکیب می‌شوند:

- پلک‌زدن (Blink): با نسبت ابعاد چشم (EAR)؛ چشم بسته می‌شود EAR افت می‌کند.
- بافت/فرکانس (Texture): صورت واقعی جزئیات ریز دارد؛ عکس چاپی یا صفحه‌ی موبایل خیر.
- حرکت سر (Motion): آدم واقعی ریزحرکت‌های غیرارادی دارد.

نتیجه‌ی ترکیبی از آستانه‌ای عبور می‌کند و چهره‌ی غیرزنده رد می‌شود:

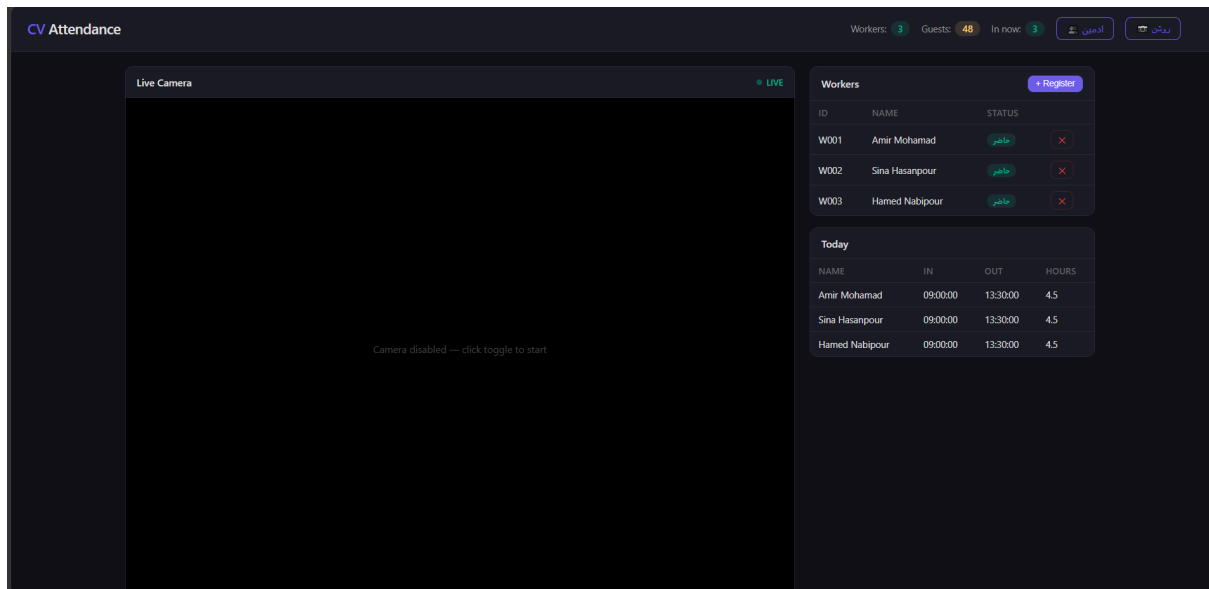
```
1 if not liveness_result.is_live:
2     continue
```

۳.۶ ردیابی چهره (Tracking)

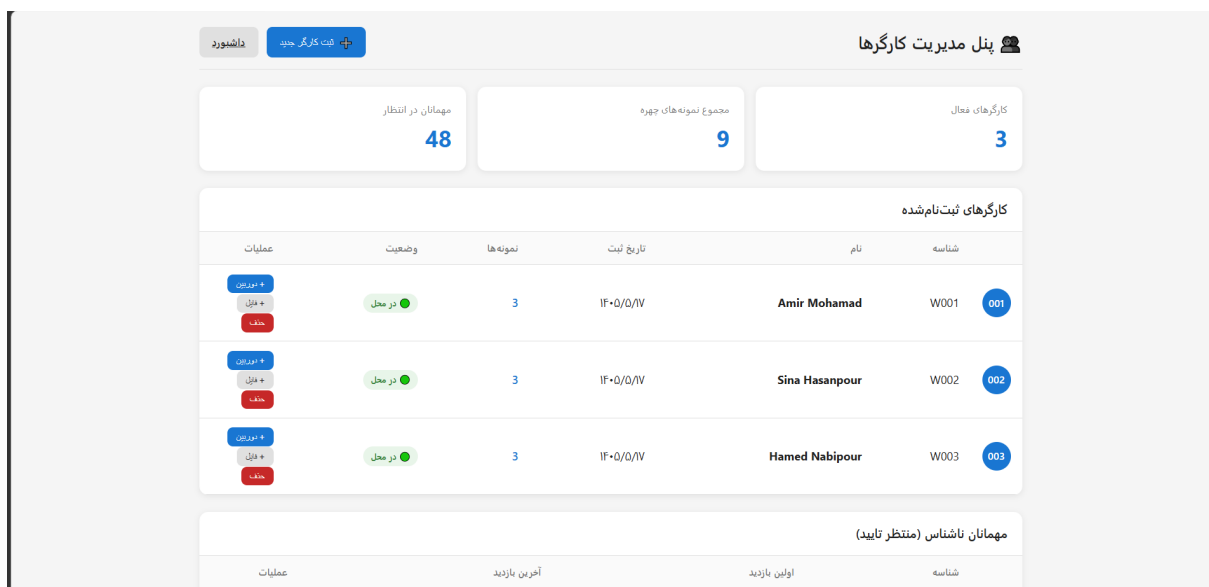
برای پایداری هویت بین فریم‌ها، از IoU برای تطبیق کادرها و Kalman Filter برای هموارسازی حرکت استفاده می‌شود. هویت نهایی با رأی‌گیری در طول چند فریم تعیین می‌شود تا از «چشم‌زدن» اسم روی صفحه جلوگیری شود.

۷ تصاویر محیط برنامه

در این بخش، تصاویر واقعی از محیط اجرای سامانه ارائه شده است.



شکل ۲: داشبورد اصلی سامانه: نمای زنده‌ی دوربین، لیست کارکنان و جدول حضور امروز



شکل ۳: پنل مدیریت (Admin): مشاهده‌ی نمونه‌های چهره، ابعاد بردارها و مدیریت کارکنان

۸ روند تیمی و تقسیم کار

این پروژه به صورت گروهی و در سه فاز اصلی انجام شد:

۱.۸ فاز صفر — طراحی معماری (کار مشترک)

هر سه نفر روی معماری سه لایه و انتخاب InsightFace/ArcFace به عنوان موتور اصلی به توافق رسیدند و اسکلت پروژه چیده شد.

۲.۸ فاز یک — سینا حسن پور (داده و منطق)

زیرساخت داده را بنا کرد: مدل های ORM، مایگريشن های Alembic، منطق ثبت ورود/خروج و محاسبه ی ساعت کار، تفکیک مهمان از کارمند و مهاجرت بردارها از ۱۲۸ به ۵۱۲ بُعدی (با پشتیبان گیری و بازسازی از روی تصاویر).

۳.۸ فاز دو — حامد نبی پور (بینایی ماشین و هوش مصنوعی)

موتور شناسایی را راه اندازی و مقاوم کرد: رفع باگ SCRFD broadcast، جداسازی فریم خام از نمایشی، حذف هم ترازای دستی اضافی، و ساخت مازول های کیفیت، زنده بودن و ردیابی.

۴.۸ فاز سه — امیرمحمد احمدی (وب، API و رابط کاربری)

سیستم را به سرویس وب تبدیل کرد: بهینه سازی هم روندی FastAPI با Thread Pool، استریم MJPEG و WebSocket، پنل ادمین و زیرساخت JWT/RBAC.

۹ نتیجه گیری و کارهای آینده

در این پروژه، یک سامانه ی حضور و غیاب خودکار مبتنی بر تشخیص چهره به صورت end-to-end پیاده سازی شد. ترکیب بردار ۵۱۲ بُعدی ArcFace، تشخیص زنده بودن و ردیابی پایدار، دقت و امنیت بالایی فراهم می کند. همچنین لایه ی وب با بهینه سازی Thread Pool به پاسخ دهی بلادرنگ دست یافته است. بهبودهای آینده:

- افزایش دقت در نورهای نامساعد و تعداد کم نمونه ها.
- خوشه بندی خودکار مهمان ها برای ارتقای دستی ساده تر.
- پشتیبانی از چند دوربین و استقرار ابری.
- اجرای مدل روی GPU برای افزایش نرخ فریم.